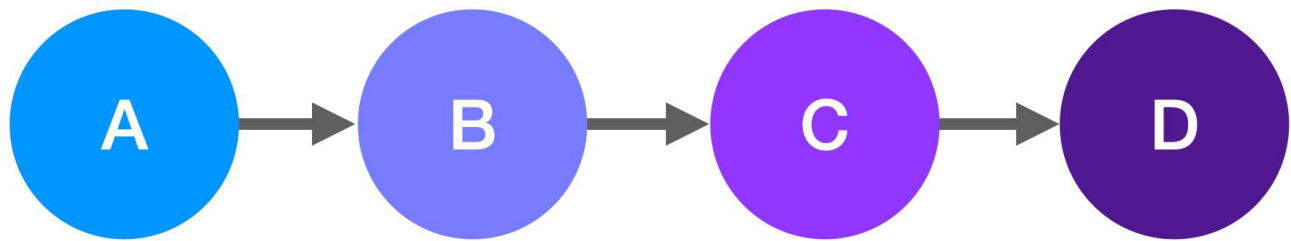




How Do Singly Linked Lists Work and How Do They Differ From Doubly Linked List?

A **linked list** is a linear data structure in which each node is connected to the next node in the sequence.

These connections create a data structure that looks like a chain of nodes, where each node stores data and a reference to the next node in the linked list.



We use these references to go from the first node to the next node and so on.

Linked lists are commonly used for implementing other data structures, such as stacks, queues, and deques. They can also be used to implement essential graph algorithms, such as depth-first search and breadth-first search.

Singly Linked Lists

A **singly linked list** is a type of linked list in which each node is connected to the next node in the sequence.

Each node is connected to the next one by storing a reference to it.

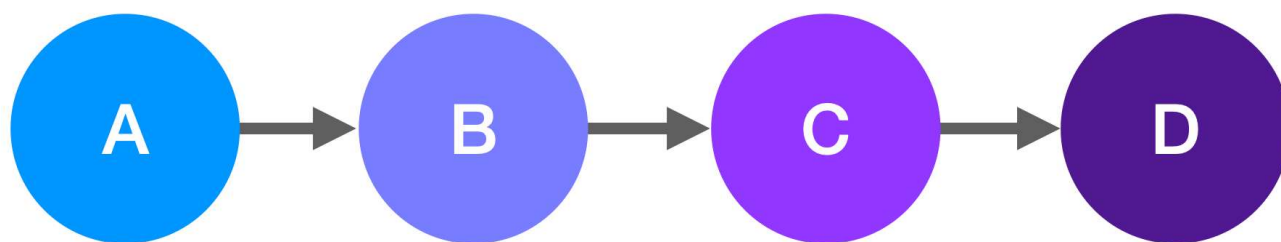
This single reference per node allows you to traverse the linked list in one direction, from start to end.

The search can only move forward, not backwards.

In this example, you would start at the head node, node A.

The **head** node is the first node in the linked list.

In a singly linked list, the head node is usually the only node that is directly accessible. This is where the search process will start when you're trying to find a specific node.



The process will start at node A, then it will continue to node B, then node C, and finally node D, the **tail** node. It may also stop before that if you implement specific logic in your code.

The **tail** node is the last node. It's used to determine when the process has reached the end of the linked list.

Inserting Nodes

One of the great things about linked lists is that they do not have a fixed size. They can be expanded or shrunk as needed by simply updating the connections between the nodes.

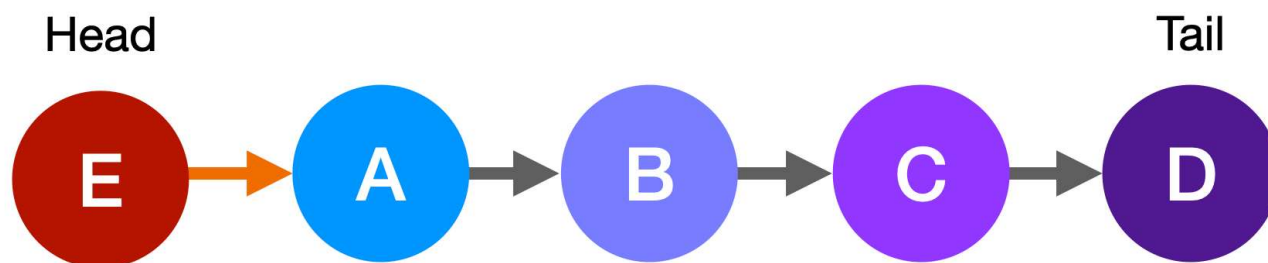
You can **insert** a node at the start, middle, and end of a linked list.

Linked lists don't necessarily need to store the nodes in a specific order. The order will be determined by the connections between the nodes.

However, if you do need to keep the nodes in a specific order for your particular use case, you can do so by implementing that logic in your code and the criteria you implement will determine if the node is inserted at the start, middle, or end.

To insert a node at the start of the linked list, you just need to create a connection between the new node and the node that used to be the head node and make the new node the head node instead.

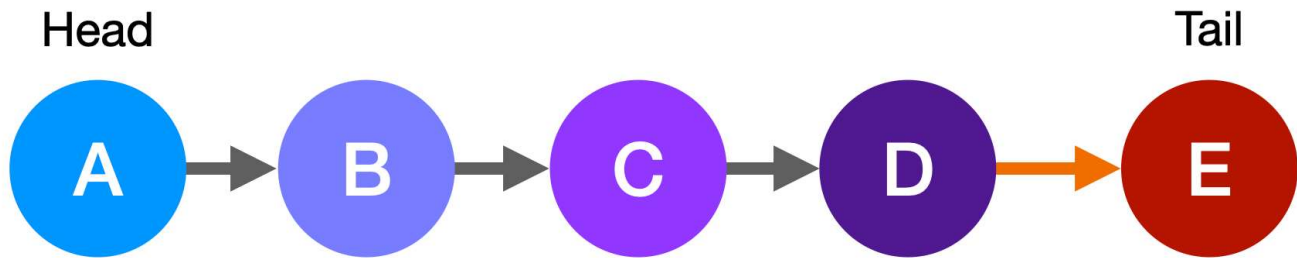
This is an example, where we insert node E at the start and make this new node the head node of the linked list.



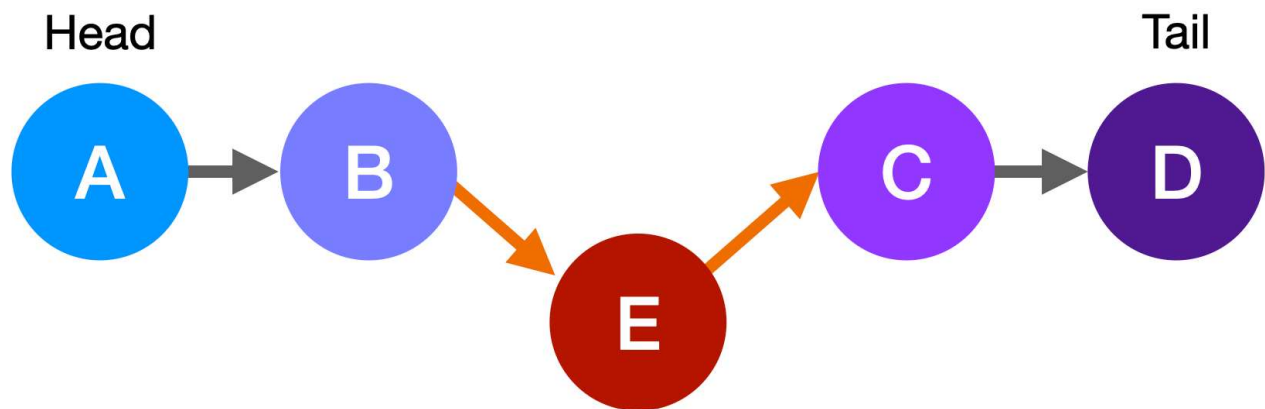
Inserting a node at the beginning of the linked list has a constant time complexity $O(1)$ because it only requires updating the reference to the head node and the connection between the new head node and the next node in the sequence.

In this example, we are inserting node E at the start of the linked list. This will work correctly. But if we wanted to keep the linked list sorted in alphabetical order, node E would have to be inserted at the end of the linked list instead.

To insert a node at the end of the linked list, first you need to reach the end and then add a connection to the new node to make it the new tail node.



If the node has to be inserted somewhere in the middle of the linked list, the connections between the nodes will have to be updated too. The previous node in the sequence should be connected to the new node and the new node should be connected to the next node, like in the following diagram.



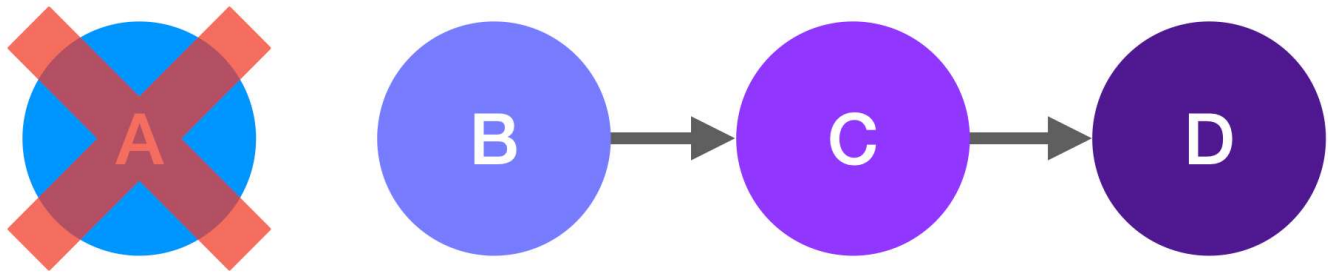
The insertion operation has a constant space complexity $O(1)$, since inserting a new node only requires creating it and updating the connections between the nodes. This operation doesn't depend on the size of the linked list itself.

Removing Nodes

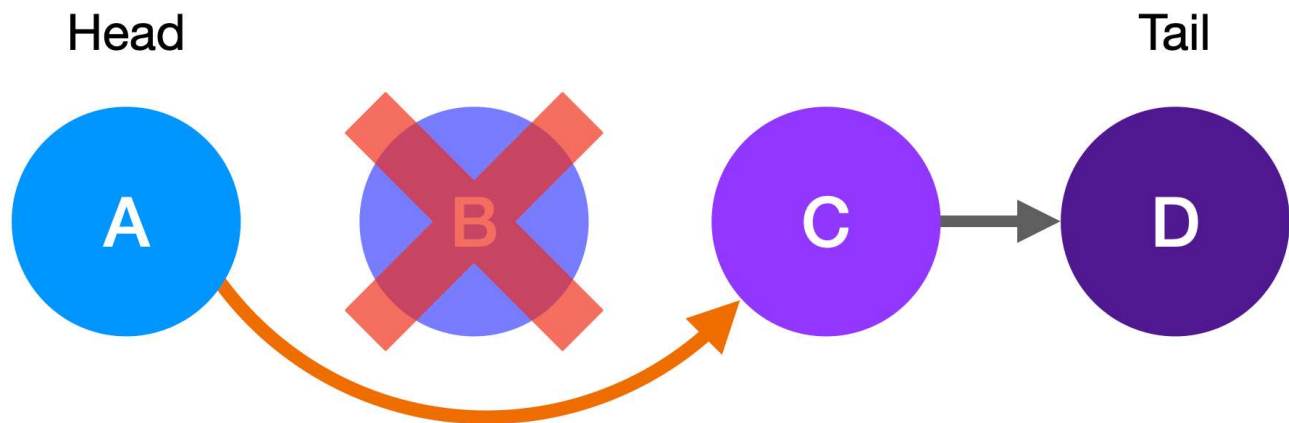
Just as you can insert nodes, you can also remove them from the start, middle, and end of the linked list.

To remove a node from the start, you need to update the reference to the head node, which should be the next node in the sequence.

This operation has a constant time complexity $O(1)$, because it only requires updating the linked list's reference to the head node.



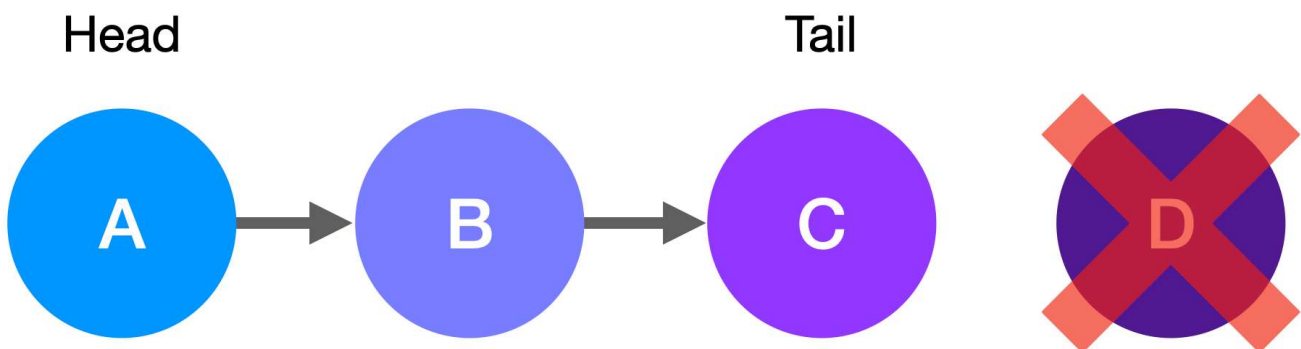
To remove a node from the middle of the linked list, you need to update the reference of the previous node to connect it to the next node in the sequence, forming a sort of "bridge" between them, as you can see in this diagram.



That will remove the node that you want to remove, in this case node B, from the sequence of connections, so it won't be reached the next time you traverse it.

To remove a node from the end of the linked list, you need to remove the connection of the previous node and make this node the new tail node. Now the linked list will end at the new tail node.

This operation has a linear time complexity $O(n)$, because first you have to reach the end of the linked list.

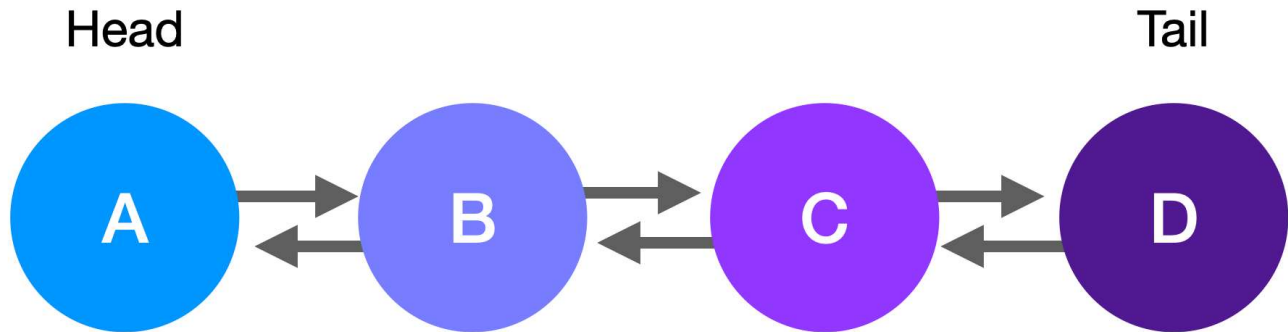


The deletion operation has a constant space complexity $O(1)$, because no additional memory is required to delete a node.



In a **doubly linked list**, each node stores two references: a reference to the next node and a reference to the previous node in the sequence.

This means that doubly linked lists can be traversed in both directions.



In this type of linked list, it's also common to keep a reference to the tail node in the linked list itself to start the traversal from the end if necessary.

That sounds great, right? They're more flexible than singly linked lists.

However, doubly linked lists do require more memory than singly linked lists because each node stores **two** references instead of one.

This is something that you should keep in mind when you're choosing the right data structure for your project.

There is a tradeoff.

The insertion and deletion operations work exactly the same. The only difference is that now you will need to update two references per node and keep track of the reference to the tail node to insert elements at the end of the doubly linked list very efficiently and start the traversal process from the back, if necessary.

Singly and doubly linked lists are essential data structures in computer science used for storing and manipulating elements in a sequential order. Understanding their differences is essential for choosing the right one for your specific application.

Questions

What is a linked list?

A data structure that stores elements in a contiguous block of memory.

A data structure where nodes are connected using references.

Correct!

A data structure that is always sorted.